

W

WAREHOUSE MANAGEMENT SYSTEM

Controle de Stages WMS

Documentação Oficial do Sistema

Sistema de gestão de armazém (WMS) para controle de ocupação de posições em stages de armazenagem. Permite cadastrar paletes, pesquisá-los por TRO ou transportadora, visualizar a ocupação em mapa visual e gerenciar a configuração de stages e transportadoras — com persistência real em banco de dados PostgreSQL.

SISTEMA

Controle de Stages WMS

VERSÃO

0.2.0

DATA DE GERAÇÃO

01 de Julho de 2026

CLASSIFICAÇÃO

Documentação Técnica

Sumário

01	Visão Geral
02	Tecnologias Utilizadas
03	Arquitetura
04	Estrutura Completa do Projeto
05	Regras de Negócio
06	Banco de Dados
07	Componentes
08	Hooks
09	Services
10	Fluxos Operacionais
11	Instalação
12	Manutenção
13	Roadmap
14	Link Oficial
15	Linguagem de Programação

01 Visão Geral

Objetivo do Sistema

O **Controle de Stages WMS** é um sistema de Warehouse Management System (WMS) que controla a ocupação de posições em áreas de armazenagem (Stages). Cada Stage possui ruas e posições organizadas em grade. O sistema permite cadastrar paletes em posições específicas, pesquisá-los por TRO ou transportadora, e visualizar a ocupação em um mapa visual.

Problema que Resolve

Em armazéns e centros de distribuição, é fundamental saber exatamente qual paleta ocupa qual posição, evitar duplicidade de ocupação, e localizar rapidamente um paleta por seu identificador (TRO) ou transportadora. Sem um sistema dedicado, esse controle é feito manualmente (planilhas ou papel), sujeito a erros, perda de informação e dificuldade de pesquisa. O Controle de Stages WMS resolve isso com:

- Mapa visual em tempo real da ocupação de cada stage.
- Garantia de que cada posição abriga no máximo um paleta (regra enforced no banco).
- Pesquisa instantânea por TRO (normalizado) ou transportadora em todos os stages.
- Persistência real em banco de dados — os dados sobrevivem a refresh, fechamento do navegador e reddeploys.
- Indicadores de ocupação (total, livres, ocupados, taxa) sempre atualizados.

Fluxo Operacional

Configuração inicial

- | Cadastrar Stages (nome, ruas, posições)
- | Cadastrar Transportadoras



Uso diário

- | Clicar em posição livre no mapa → cadastrar palete (transportadora + TR0 + remessa)
- | Clicar em posição ocupada → visualizar / editar / liberar
- | Pesquisar por TR0 ou transportadora → visualizar no mapa



Gestão

- Excluir stages/transportadoras não utilizados
- Acompanhar indicadores de ocupação

Público-alvo

Operadores de armazém, supervisores de logística e equipes de expedição que precisam controlar a ocupação de posições em tempo real, com pesquisa rápida e visualização clara do estado do armazém.

02 Tecnologias Utilizadas

Todas as tecnologias que compõem o projeto, com sua respectiva função:

React 18.3

Framework de interface. Componentes funcionais com hooks. Renderização declarativa do mapa, modais e painéis.

TypeScript 5.5

Tipagem estática sobre JavaScript. Garante contratos de dados em tempo de compilação e reduz bugs em runtime.

Vite 5.3

Bundler e dev server. Build otimizado com HMR instantâneo em desenvolvimento e tree-shaking em produção.

Supabase JS SDK 2.110

Cliente PostgreSQL gerenciado. Fornece API REST automática, RLS e autenticação. Usado aqui para persistência real dos dados.

PostgreSQL

Banco de dados relacional gerenciado pelo Supabase. Armazena stages, carriers e pallets com constraints, índices e triggers.

lucide-react 0.383

Biblioteca de ícones SVG. Usada em botões, cabeçalhos e indicadores visuais (mapa, pacote, caminhão, etc.).

HTML5 + CSS3 inline

Estrutura semântica e estilização via design tokens (cores, espaçamentos, raios) centralizados em `styles/tokens.ts`.

npm

Gerenciador de pacotes. Instala dependências, executa scripts (`dev` , `build` , `preview`).

Nota: O projeto não utiliza TailwindCSS. A estilização é feita via design tokens em TypeScript (`src/styles/tokens.ts`) aplicados como estilos inline, garantindo um único ponto de verdade visual.

03 Arquitetura

A arquitetura segue o princípio da Responsabilidade Única (SRP) em camadas bem definidas. Componentes nunca acessam o banco diretamente; regras de negócio vivem em hooks; lógica de pesquisa é pura; e a persistência é isolada em um único service.

Camada de Apresentação
components / pages — React + estilos inline



Camada de Estado e Regras de Negócio
hooks — useWmsData, useToast, useMapHighlight, useDebounceValue



Camada de Serviços
services — storage.ts (persistência) · search.ts (pesquisa pura)



Camada de Dados
Supabase / PostgreSQL — tables: stages, carriers, pallets

Comunicação entre Componentes

Os componentes se comunicam via **props** (dados de entrada e callbacks). O estado global da aplicação vive no hook `useWmsData`, instanciado em `StagesPage`. Modais e painéis recebem dados via props e notificam alterações via callbacks (`onSave`, `onRelease`, `onViewOnMap`).

Circulação de Dados

Na inicialização, `useWmsData` chama `wmsStorage.loadAll()` que consulta as três tabelas em paralelo e popula o estado local. A partir daí, toda escrita (criar/editar/liberar/excluir) atualiza o estado local e persiste no banco imediatamente. Derivados (`occupiedByStage`, `indicators`) são memoizados e recalculam apenas quando a fonte muda.

Como funciona o Banco

O Supabase expõe um endpoint REST sobre PostgreSQL. O cliente usa a `anon key` (segura para operações permitidas pelo RLS). As tabelas `stages`, `carriers` e `pallets` têm Row Level Security habilitado com políticas que permitem CRUD completo para `anon` e `authenticated` (single-tenant, sem autenticação por enquanto). Uma constraint `UNIQUE(stage_id, street, position)` garante no nível do banco que duas posições não sejam ocupadas pelo mesmo palete.

Como funciona o Mapa

O `StageMap` gera a grade dinamicamente a partir da configuração do Stage: colunas = ruas, linhas = posições (sempre de cima para baixo). Cada célula é um botão circular colorido: verde (livre) ou vermelho (ocupado, com ícone de pacote). Posições destacadas pela pesquisa recebem borda azul e glow. O componente é memoizado com `React.memo` para evitar re-renderizações quando apenas outros stages mudam.

Como funciona a Pesquisa

O `SearchPanel` usa `useDebounceValue` (350ms) para evitar consultar a cada tecla. A lógica pura em `search.ts` normaliza o TRO (remove prefixo, espaços, zeros à esquerda) para correspondência exata, ou filtra por parte do nome da transportadora (case-insensitive). A pesquisa sempre considera todos os stages. Resultados são ordenados por Stage → Rua → Posição e agrupados por Stage.

04 Estrutura Completa do Projeto

```
project/
├─ index.html           HTML raiz
├─ package.json         Dependências e scripts
├─ tsconfig.json        Configuração TypeScript
├─ vite.config.ts        Configuração Vite
├─ .env                 Variáveis de ambiente (Supabase)
├─ docs/                Documentação gerada
├─ supabase/
│   └─ migrations/
│       └─ 001_create_wms_tables.sql  Schema do banco
└─ src/
    ├─ main.tsx          Entry point React
    ├─ App.tsx           Componente raiz
    ├─ pages/
    │   └─ StagesPage.tsx  Tela operacional principal
    ├─ components/
    │   ├─ StageMap.tsx    Mapa visual de posições
    │   ├─ SearchPanel.tsx Painel de pesquisa
    │   ├─ ConfigPanel.tsx Modal de configuração
    │   ├─ PositionModal.tsx Modal de posição
    │   ├─ CarrierCombobox.tsx Combobox de transportadoras
    │   └─ ui/             Design System
    │       ├─ Button.tsx
    │       ├─ Field.tsx
    │       ├─ Modal.tsx
    │       ├─ Row.tsx
    │       ├─ Toast.tsx
    │       └─ index.ts
    ├─ hooks/
    │   ├─ useWmsData.ts    Hook central de dados
    │   ├─ useToast.ts      Notificações
    │   ├─ useMapHighlight.ts Destaque no mapa
    │   └─ useDebounceValue.ts Debounce
    ├─ services/
    │   ├─ storage.ts       Persistência (Supabase)
    │   └─ search.ts        Lógica de pesquisa
    ├─ styles/
    │   └─ tokens.ts        Design tokens
    ├─ types/
    │   └─ index.ts         Tipos de domínio
    └─ utils/
        ├─ uid.ts           Gerador de UUIDs
        └─ tro.ts           Normalização de TR0
```


Responsabilidade de cada pasta

PASTA	RESPONSABILIDADE
<code>src/pages</code>	Telas completas da aplicação. Hoje apenas <code>StagesPage</code> .
<code>src/components</code>	Componentes reutilizáveis de UI específica do domínio (mapa, painéis, modais).
<code>src/components/ui</code>	Design System genérico (Button, Modal, Field, Toast, Row).
<code>src/hooks</code>	Lógica de estado e regras de negócio reutilizável.
<code>src/services</code>	Camada de acesso a dados (storage) e lógica pura (search).
<code>src/styles</code>	Design tokens centralizados (cores, espaçamentos, raios).
<code>src/types</code>	Tipos de domínio compartilhados.
<code>src/utils</code>	Funções utilitárias puras (uid, tro).
<code>supabase/migrations</code>	Scripts SQL versionados do banco de dados.
<code>docs</code>	Documentação técnica e SRS.

05 Regras de Negócio

Todas as regras implementadas, com sua localização no código:

ID	REGRA	IMPLEMENTAÇÃO
RN-01	Nunca permitir dois paletes na mesma Stage+Rua+Posição	<code>savePallet</code> verifica conflito + constraint UNIQUE no banco
RN-02	Transportadora é sempre selecionada da lista (nunca digitação livre)	<code>CarrierCombobox</code>
RN-03	Nome de Stage é único (case-insensitive)	<code>createStage</code> valida + UNIQUE no banco
RN-04	Nome de Transportadora é único (case-insensitive)	<code>createCarrier</code> valida + UNIQUE no banco
RN-05	Não excluir Stage com paletes vinculados	<code>deleteStage</code> bloqueia
RN-06	Não excluir Transportadora com paletes vinculados	<code>deleteCarrier</code> bloqueia
RN-07	TRO é normalizado (apenas dígitos) para pesquisa	<code>normalizeTRODigits</code>
RN-08	Pesquisa por TRO remove zeros à esquerda para comparação	<code>troSearchKey</code>
RN-09	Resultados ordenados por Stage → Rua → Posição	<code>searchPallets</code> sort
RN-10	Colunas do mapa = Ruas, linhas = Posições (sempre de cima para baixo)	<code>StageMap</code>
RN-11	Destaque do mapa remove-se automaticamente após 3,2s	<code>useMapHighlight</code>
RN-12	Debounce de pesquisa entre 300-500ms	<code>useDebounceValue</code> (350ms)
RN-13	Modal fecha automaticamente após salvar com sucesso; permanece aberto em erro	<code>StagesPage.handleSavePallet</code>
RN-14	Nunca usar <code>alert()</code> do navegador — sempre Toast	<code>Toast</code> component

ID	REGRA	IMPLEMENTAÇÃO
RN-15	IDs sempre UUIDs válidos (RFC 4122)	<code>uid()</code> usa <code>crypto.randomUUID()</code>

06 Banco de Dados

Banco de dados **PostgreSQL** gerenciado pelo Supabase. A migration `001_create_wms_tables.sql` cria todas as tabelas, constraints, índices, trigger e políticas RLS.

Tabela `stages`

COLUNA	TIPO	CONSTRAINTS
<code>id</code>	uuid	PK DEFAULT <code>gen_random_uuid()</code>
<code>name</code>	text	UNIQUE NOT NULL
<code>streets</code>	integer	NOT NULL, CHECK <code>> 0</code>
<code>positions</code>	integer	NOT NULL, CHECK <code>> 0</code>
<code>created_at</code>	timestampz	DEFAULT <code>now()</code>

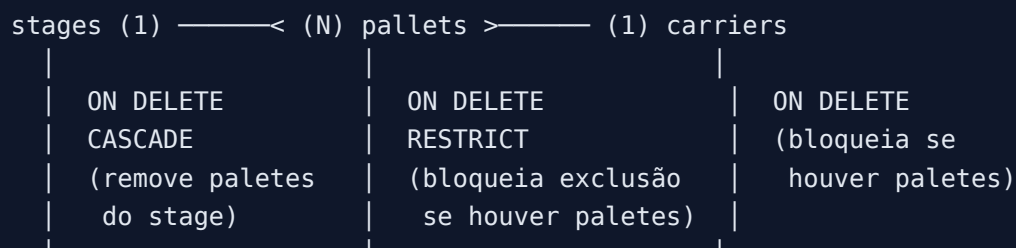
Tabela `carriers`

COLUNA	TIPO	CONSTRAINTS
<code>id</code>	uuid	PK DEFAULT <code>gen_random_uuid()</code>
<code>name</code>	text	UNIQUE NOT NULL
<code>active</code>	boolean	NOT NULL DEFAULT <code>true</code>
<code>created_at</code>	timestampz	DEFAULT <code>now()</code>

Tabela `pallets`

COLUNA	TIPO	CONSTRAINTS
<code>id</code>	uuid	PK DEFAULT <code>gen_random_uuid()</code>
<code>stage_id</code>	uuid	NOT NULL, FK → <code>stages(id)</code> ON DELETE CASCADE
<code>carrier_id</code>	uuid	NOT NULL, FK → <code>carriers(id)</code> ON DELETE RESTRICT
<code>street</code>	integer	NOT NULL, CHECK > 0
<code>position</code>	integer	NOT NULL, CHECK > 0
<code>tro</code>	text	NOT NULL
<code>shipment</code>	text	DEFAULT ''
<code>created_at</code>	timestampz	DEFAULT <code>now()</code>
<code>updated_at</code>	timestampz	DEFAULT <code>now()</code>

Relacionamentos



Constraint Única

`UNIQUE (stage_id, street, position)` — garante a regra de negócio RN-01 no nível do banco: é impossível que dois paletes ocupem a mesma posição, mesmo que a validação no cliente falhe.

Índices

ÍNDICE	TABELA/COLUNA	FINALIDADE
IDX <code>idx_pallets_stage_id</code>	<code>pallets.stage_id</code>	Acelera filtro por Stage
IDX <code>idx_pallets_carrier_id</code>	<code>pallets.carrier_id</code>	Acelera filtro por transportadora
IDX <code>idx_pallets_tro</code>	<code>pallets.tro</code>	Acelera pesquisa por TRO

Trigger

`update_pallets_updated_at` — atualiza automaticamente a coluna `updated_at` em qualquer UPDATE da tabela `pallets`, via função PL/pgSQL `update_updated_at_column()`.

RLS (Row Level Security)

Todas as tabelas têm RLS habilitado. Políticas permitem CRUD completo para `anon` e `authenticated` (single-tenant, sem autenticação por enquanto). Quando autenticação for adicionada, basta trocar `USING (true)` por `USING (auth.uid() = user_id)`.

Fluxo de Gravação

Componente

- `useWmsData` (valida regra de negócio)
- `setEstado local` (atualização otimista)
- `wmsStorage.saveXxx()`
- Supabase (upsert/delete via REST)
- PostgreSQL

Fluxo de Leitura

- `useWmsData` (`useEffect` na inicialização)
 - `wmsStorage.loadAll()`
 - Supabase (SELECT em paralelo: `stages`, `carriers`, `pallets`)
 - mapeia rows `snake_case` → `camelCase`
 - `setEstado local`
 - componentes renderizam

07 Componentes

StagesPage.tsx

Tela operacional principal. Renderiza cabeçalho (título + sincronização + botão Configuração), indicadores (total/livres/ocupados/taxa), abas de Stages, o mapa do Stage ativo, o painel de pesquisa e os modais. Orquestra `useWmsData`, `useToast` e `useMapHighlight`.

StageMap.tsx

Mapa visual em grade. Colunas = Ruas, linhas = Posições. Cada célula é um botão circular: verde (livre) ou vermelho (ocupado, com ícone de pacote). Posições destacadas pela pesquisa recebem borda azul. Memoizado com `React.memo`.

SearchPanel.tsx

Painel de pesquisa lateral. Alternância entre TRO e Transportadora. Campo com debounce de 350ms. Resultados agrupados por Stage, cada grupo com botão "Visualizar no mapa".

ConfigPanel.tsx

Modal de configuração com duas abas: Stages (criar com nome/ruas/posições, listar e excluir) e Transportadoras (criar e excluir). Puramente apresentacional — validações em `useWmsData`.

PositionModal.tsx

Modal único de posição com três modos: cadastro (posição livre), visualização (posição ocupada, leitura) e edição (posição ocupada, edição). Inclui botão "Liberar posição". Fecha automaticamente após salvar com sucesso.

CarrierCombobox.tsx

Combobox pesquisável para seleção de transportadora. Filtra por nome, fecha ao clicar fora, mostra check no item selecionado. Garante que a transportadora seja sempre selecionada da lista (RN-02).

ui/Button.tsx

Botão padronizado com 5 variantes: primary, secondary, danger, success, ghost. Inclui ícone opcional, estado disabled e efeito de hover.

ui/Field.tsx

Wrapper de rótulo + campo. Inclui `TextInput` com foco visual (borda azul ao focar).

ui/Modal.tsx

Estrutura base de modal: overlay, título, descrição, corpo, rodapé. Fecha ao clicar no overlay ou no botão X.

ui/Row.tsx

Linha rótulo/valor reutilizável para telas de detalhe.

ui/Toast.tsx

Notificação temporária fixa no rodapé. Variantes success e error. Substitui `alert()` em todo o sistema (RN-14).

08 Hooks

useWmsData

Hook central de dados. Concentra TODAS as regras de negócio de leitura/escrita. Carrega dados do Supabase na inicialização. Toda escrita atualiza o estado local e persiste no banco imediatamente.

Expõe:

- **Estado:** `loading`, `stages`, `carriers`, `pallets`, `lastSync`, `occupiedByStage`, `indicators`.
- **Operações:** `createStage`, `deleteStage`, `createCarrier`, `deleteCarrier`, `savePallet`, `releasePallet`, `refresh`.

useToast

Gerencia notificações temporárias. `showToast(message, type)` exibe uma toast que desaparece após 2800ms. Cancela toast anterior se uma nova for exibida.

useMapHighlight

Gerencia o destaque de posições no mapa. `highlightOnStage(stageId, positionKeys)` destaca posições em azul e remove o destaque após 3200ms. Inclui limpeza do timer no desmontar.

useScrollToHighlight

Centraliza a primeira posição destacada na tela via `scrollIntoView`.

useDebounceValue

Atrasa a propagação de um valor em `delay` ms. Usado pela pesquisa para evitar consultar a cada tecla (350ms).

09 Services

`services/storage.ts`

Camada de persistência usando Supabase. Isola toda a lógica de acesso ao banco. Mapeia campos snake_case (DB) para camelCase (TS).

Funções:

- `loadAll()` — carrega stages, carriers e pallets em paralelo (Promise.all).
- `saveStages(stages)` — sincroniza a lista de stages (upsert + delete diff).
- `saveCarriers(carriers)` — sincroniza a lista de carriers.
- `savePallets(pallets)` — sincroniza a lista de pallets.

`services/search.ts`

Lógica de pesquisa pura (sem estado, sem side-effects).

Funções:

- `searchPallets({ type, query, pallets, stages, carriers })` — pesquisa por TRO (correspondência exata após normalização) ou transportadora (parte do nome, case-insensitive). Sempre pesquisa em todos os Stages. Retorna resultados ordenados por Stage → Rua → Posição.
- `groupResultsByStage(results)` — agrupa resultados por Stage, preservando a ordem.

10 Fluxos Operacionais

Cadastrar Paleta

1. Usuário clica em posição livre no mapa
2. StagesPage abre PositionModal (modo cadastro)
3. Usuário seleciona transportadora (combobox), digita TR0, remessa opcional
4. Usuário clica "Salvar"
5. PositionModal valida campos obrigatórios
6. StagesPage.handleSavePallet → useWmsData.savePallet
7. savePallet verifica conflito de posição (RN-01)
8. Cria registro, atualiza estado local, persiste no Supabase
9. Em sucesso: fecha modal automaticamente (RN-13)
10. Toast de sucesso exibido

Editar Paleta

1. Usuário clica em posição ocupada no mapa
2. PositionModal abre em modo visualização
3. Usuário clica "Editar" → modal muda para modo edição
4. Usuário altera transportadora/TR0/remessa
5. Clica "Salvar"
6. savePallet (modo edição) atualiza o registro no estado e no banco
7. Trigger update_pallets_updated_at atualiza updated_at
8. Modal fecha, Toast exibido

Liberar Posição

1. Usuário clica em posição ocupada
2. PositionModal abre em modo visualização
3. Usuário clica "Liberar posição"
4. useWmsData.releasePallet remove o paleta do estado
5. wmsStorage.savePallets persiste (delete diff)
6. Modal fecha, Toast de sucesso exibido
7. Posição volta a ficar verde (livre) no mapa

Pesquisar

1. Usuário seleciona tipo (TRO ou Transportadora) no SearchPanel
2. Digita query
3. `useDebounceValue` aguarda 350ms (RN-12)
4. `searchPallets` normaliza query e filtra pallets em todos os Stages
5. Resultados ordenados por Stage → Rua → Posição (RN-09)
6. `groupResultsByStage` agrupa por Stage
7. Grupos renderizados com botão "Visualizar no mapa"

Visualizar no Mapa

1. Usuário clica "Visualizar no mapa" em um grupo de resultados
2. `StagesPage.handleViewOnMap` troca o Stage ativo
3. `useMapHighlight.highlightOnStage` destaca as posições em azul
4. `useScrollToHighlight` rola a tela para a primeira posição
5. Destaque remove-se automaticamente após 3,2s (RN-11)

11 Instalação

Como Instalar

```
# Clonar o repositório e instalar dependências
npm install
```

Como Executar (desenvolvimento)

```
# Inicia o Vite dev server com HMR
npm run dev

# A aplicação abre em http://localhost:5173
```

Como Publicar (build de produção)

```
# Gera o build otimizado em dist/
npm run build

# Pré-visualiza o build de produção localmente
npm run preview
```

Como Configurar o Ambiente

O arquivo `.env` na raiz do projeto deve conter as variáveis do Supabase:

```
VITE_SUPABASE_URL=https://<projeto>.supabase.co
VITE_SUPABASE_ANON_KEY=<chave-anon>
```

Essas variáveis são lidas em `src/services/storage.ts` via `import.meta.env`. Os tipos estão declarados em `src/vite-env.d.ts`.

Como Configurar o Banco

A migration `supabase/migrations/001_create_wms_tables.sql` cria todas as tabelas, índices, constraints, trigger e políticas RLS. Aplicar via Supabase MCP ou painel do Supabase. Após

aplicada, o sistema está pronto para uso — sem dados mock, sem LocalStorage, sem SessionStorage.

12 Manutenção

Como Adicionar Novas Funcionalidades

TIPO	ONDE	COMO
Nova tela	<code>src/pages/</code>	Criar componente, usar hooks existentes (<code>useWmsData</code>).
Novo componente de UI	<code>src/components/ui/</code>	Criar, exportar em <code>index.ts</code> .
Nova regra de negócio	<code>src/hooks/useWmsData.ts</code>	Adicionar validação na operação correspondente (nunca em componentes).
Nova tabela	<code>supabase/migrations/</code>	Criar migration, adicionar mapper em <code>storage.ts</code> , tipo em <code>types/index.ts</code> .
Nova pesquisa	<code>src/services/search.ts</code>	Estender lógica pura (sem side-effects).
Novo token visual	<code>src/styles/tokens.ts</code>	Adicionar cor/espacamento/raio. Nunca usar valores soltos.

Boas Práticas

- Componentes nunca acessam o banco diretamente — sempre via `useWmsData` ou `wmsStorage` .
- Regras de negócio ficam em `hooks/services`, nunca em componentes.
- Design tokens em `styles/tokens.ts` — nunca cores/espacamentos soltos.
- IDs sempre gerados por `uid()` (UUIDs válidos para o banco).
- TRO normalizado apenas via `utils/tro.ts` .
- Nunca usar `alert()` — sempre `Toast` .
- Nunca usar `LocalStorage/SessionStorage` — sempre Supabase.
- Memoizar derivados com `useMemo` e componentes pesados com `React.memo` .

13 Roadmap

Fase 2 — Autenticação

- Login com email/senha (Supabase Auth).
- Multi-tenant (dados isolados por organização).
- RLS com `auth.uid()` em vez de `USING (true)`.

Fase 3 — Auditoria

- Tabela `pallet_history` com triggers de insert/update/delete.
- Tela de histórico por posição.
- Rastreabilidade completa de alterações.

Fase 4 — Dashboard

- Gráficos de ocupação temporal.
- Tendências de uso por Stage.
- Exportação CSV/PDF.

Fase 5 — Real-time

- Supabase subscriptions para sincronização entre clientes.
- Atualização automática quando outro operador cadastra/libera um palete.

Fase 6 — Qualidade

- Testes unitários (Vitest) para services e hooks.
- Testes de integração para o banco.
- CI/CD com GitHub Actions.

Link Oficial da Aplicação

Acesse o sistema publicado em produção:

<https://conceicaoabracoquent-q2yk.bolt.host/>



Nome do sistema: Controle de Stages WMS

URL oficial: <https://conceicaoabracoquent-q2yk.bolt.host/>

Aponte a câmera do seu smartphone para o QR Code acima para acessar diretamente a aplicação publicada.

15 Linguagem de Programação

Stack tecnológico completo que compõe o projeto, organizado por camada:

Frontend

TECNOLOGIA	VERSÃO	FINALIDADE
React	18.3	Framework de UI baseado em componentes. Renderização declarativa, hooks, virtual DOM.
TypeScript	5.5	Superset tipado de JavaScript. Compila para JS, garantindo contratos em tempo de compilação.
HTML5	—	Linguagem de marcação semântica. Estrutura base em <code>index.html</code> .
CSS3	—	Estilização via estilos inline e design tokens. Sem TailwindCSS — tokens centralizados em TS.
JSX/TSX	—	Extensão de sintaxe que permite HTML em JavaScript/TypeScript.

Build e Tooling

TECNOLOGIA	VERSÃO	FINALIDADE
Vite	5.3	Bundler e dev server. HMR instantâneo, build otimizado com Rollup, tree-shaking.
@vitejs/plugin-react	4.3	Plugin Vite para suporte a JSX/TSX e Fast Refresh.
typescript (tsc)	5.5	Compilador TypeScript usado no <code>tsc -b</code> antes do build do Vite.

Banco de Dados

TECNOLOGIA	FINALIDADE
PostgreSQL	Banco de dados relacional gerenciado pelo Supabase. Armazena stages, carriers e pallets com constraints, índices, triggers e RLS.
Supabase	Backend-as-a-Service sobre PostgreSQL. Fornece API REST automática, RLS, auth e realtime. Cliente JS: @supabase/supabase-js 2.110.

Gerenciador de Pacotes

TECNOLOGIA	FINALIDADE
npm	Gerenciador de pacotes do Node.js. Instala dependências, executa scripts (<code>dev</code> , <code>build</code> , <code>preview</code>).

Bibliotecas de Apoio

BIBLIOTECA	VERSÃO	FINALIDADE
lucide-react	0.383	Biblioteca de ícones SVG. Usada em botões, cabeçalhos, mapa e indicadores.
react-dom	18.3	Renderer React para o DOM. Necessário para o React funcionar no navegador.
@types/react, @types/react-dom	18.3	Definições de tipo TypeScript para React e ReactDOM.

Resumo Executivo

O projeto é uma **Single Page Application (SPA)** escrita em **TypeScript** sobre **React 18**, com build via **Vite** e persistência em **PostgreSQL** gerenciado pelo **Supabase**. O gerenciador de pacotes é **npm**. Não utiliza TailwindCSS — a estilização é feita via design tokens em TypeScript aplicados como estilos inline, garantindo um único ponto de verdade visual.